

Welcome To Mobile Monetization Pro

Introduction

Welcome to **Mobile Monetization Pro**, the ultimate toolkit designed to help you effortlessly monetize your mobile games. Whether you're a solo developer, part of a small team, or a large studio, Mobile Monetization Pro simplifies the integration of essential monetization features.

Mobile Monetization Pro offers a user-friendly, powerful solution for handling in-app purchases (IAPs), ads, and rewards, all from one package. Say goodbye to the headache of managing complex integrations, and say hello to a seamless experience that allows you to focus on what matters most—creating amazing games.

Why Choose Mobile Monetization Pro?

Created by developers, for developers, Mobile Monetization Pro was built with ease of use and flexibility in mind. We know the challenges you face when integrating monetization features into your games, and we've created the toolkit to solve those challenges.

Don't just take our word for it—Mobile Monetization Pro is trusted by hundreds of developers, helping them easily integrate ads, IAPs and more with minimal effort.

Features

Ads Integration Made Easy

Seamlessly integrate the ad networks of your choice into your mobile game with full mediation support:

- **Admob** (*Mediation Supported*)
- **AppLovin** (*Mediation Supported*)
- **LevelPlay** (*Mediation Supported*)
- **IronSource**
- **Unity Ads**

- **LiftOff (Vungle)**
-

In-App Purchases (IAP)

Easily implement IAPs and show product prices in the player's local currency:

- Supports **Consumable**, **Non-Consumable**, and **Subscription** items
 - Displays IAP prices in **local currencies**
 - Automatically detects **refunds** for Non-Consumables and Subscriptions
 - Resets premium content access when refunded to ensure fair use
-

Compliance Ready (GDPR & ATT)

Stay compliant with regional privacy laws:

- **App Tracking Transparency (ATT)** for iOS
 - **GDPR Consent** using Google UMP SDK
-

Cross Promotion System

Promote other games in your catalog with built-in cross-promotion tools to drive engagement and revenue.

User Review Prompt System

Prompt users to leave reviews for your game, improving visibility and trust on app stores.

Remove Ads Functionality

Give players the option to enjoy an **ad-free experience** with a simple purchase.

Local Push Notifications

Customizable notification system built for Unity:

- Adjustable **repeat cycles** (1–3)

- Reset notifications on app launch
 - Schedule **unique messages** at various intervals
 - Full customization for:
 - Title, subtitle (iOS), and body
 - Small and large icons
 - Improved Android notification channel registration
 - Clear script comments for easy setup
-

Remote Push Notifications

Send real-time messages with **Firebase Cloud Messaging (FCM)** integration.

In-App Update System (Android)

Let users **update your game from inside the app**, without needing to visit the Play Store.

Take Game Screenshots

Capture clean game screenshots directly from the **Unity Editor**.

Firebase Analytics Integration

Track player behavior and engagement using **Firebase Analytics**.

No Internet Connectivity Checker

- Show a **"No Internet Connection"** popup when offline
 - Resume ad display once internet is restored
-

Automatically Update System Properties

Avoid manual errors with auto-updates:

- **Android:**
 - Update `AndroidManifest.xml` and Gradle properties with one click

- **iOS:**
 - Auto-update `Info.plist` for LevelPlay integration
-

No Coding Required

Beginner-friendly! Drag-and-drop workflows let you get started without writing code.

Comprehensive Documentation & Tutorials

Learn fast with detailed written guides and step-by-step **video tutorials**.

Supported Advertisers & Mediation

- Admob
 - AppLovin
 - LevelPlay
 - IronSource
 - Unity Ads
 - LiftOff (Vungle)
-

Supported Ad Types

- Banner
 - Interstitial
 - Rewarded Video
 - Rewarded Interstitial
 - App Open
 - Native
-

Supported Platforms

- Android
- iOS

 **Thank You**

Thanks for choosing **Mobile Monetization Pro!**

Admob Ads Part-1

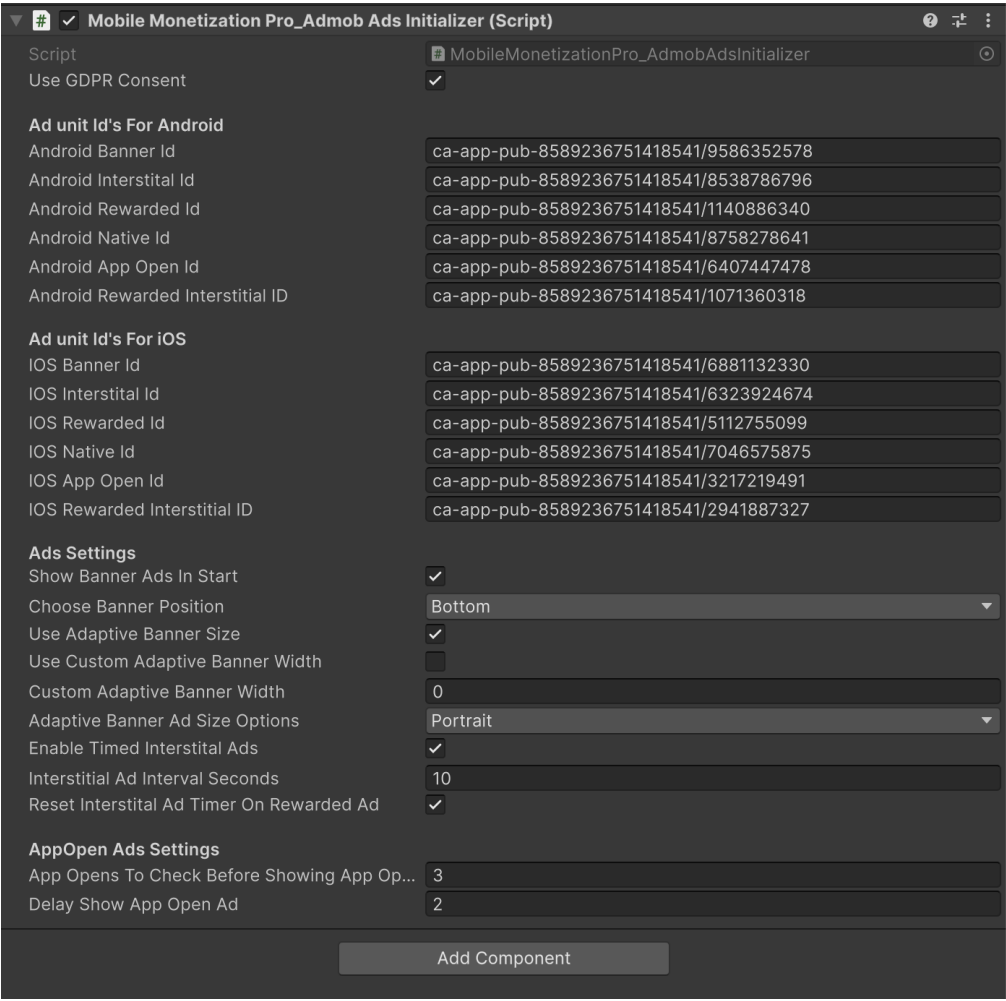
Introduction

In this video, we take the first step in integrating **Admob Ads** into your mobile game using **Mobile Monetization Pro**.

You'll learn how to:

- Set up your Admob account.
- Configure Admob SDK in Unity.
- Integrate and test ads.

Admob Ads_INITIALIZER



Script: MobileMonetizationPro_AdmobAdsInitializer

Use GDPR Consent: ☒

Ad unit Id's For Android

Android Banner Id	ca-app-pub-8589236751418541/9586352578
Android Interstitial Id	ca-app-pub-8589236751418541/8538786796
Android Rewarded Id	ca-app-pub-8589236751418541/1140886340
Android Native Id	ca-app-pub-8589236751418541/8758278641
Android App Open Id	ca-app-pub-8589236751418541/6407447478
Android Rewarded Interstitial ID	ca-app-pub-8589236751418541/1071360318

Ad unit Id's For iOS

IOS Banner Id	ca-app-pub-8589236751418541/6881132330
IOS Interstitial Id	ca-app-pub-8589236751418541/6323924674
IOS Rewarded Id	ca-app-pub-8589236751418541/5112755099
IOS Native Id	ca-app-pub-8589236751418541/7046575875
IOS App Open Id	ca-app-pub-8589236751418541/3217219491
IOS Rewarded Interstitial ID	ca-app-pub-8589236751418541/2941887327

Ads Settings

Show Banner Ads In Start	☒
Choose Banner Position	Bottom
Use Adaptive Banner Size	☒
Use Custom Adaptive Banner Width	☐
Custom Adaptive Banner Width	0
Adaptive Banner Ad Size Options	Portrait
Enable Timed Interstitial Ads	☒
Interstitial Ad Interval Seconds	10
Reset Interstitial Ad Timer On Rewarded Ad	☒

AppOpen Ads Settings

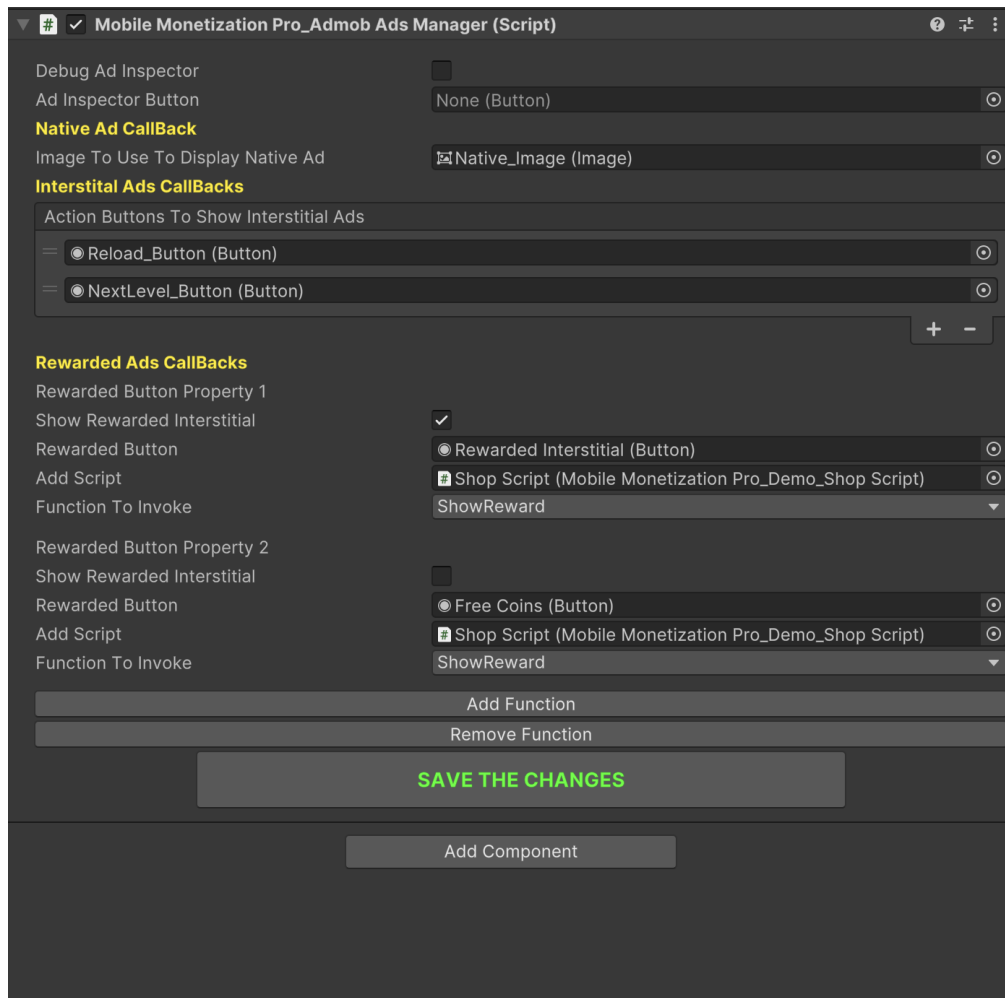
App Opens To Check Before Showing App Op...	3
Delay Show App Open Ad	2

Add Component

Fields	Info
UseGDPRConsent	Enable or disable GDPR consent support.
AndroidBannerId	Android AdMob Banner Ad Unit ID.
AndroidInterstitialId	Android AdMob Interstitial Ad Unit ID.
AndroidRewardedId	Android AdMob Rewarded Video Ad Unit ID.
AndroidNativeId	Android AdMob Native Ad Unit ID.
AndroidAppOpenId	Android AdMob App Open Ad Unit ID.
AndroidRewardedInterstitialID	Android AdMob Rewarded Interstitial Ad Unit ID.
IOSBannerId	iOS AdMob Banner Ad Unit ID.
IOSInterstitialId	iOS AdMob Interstitial Ad Unit ID.
IOSRewardedId	iOS AdMob Rewarded Video Ad Unit ID.
IOSNativeId	iOS AdMob Native Ad Unit ID.
IOSAppOpenId	iOS AdMob App Open Ad Unit ID.
IOSRewardedInterstitialID	iOS AdMob Rewarded Interstitial Ad Unit ID.
ShowBannerAdsInStart	Automatically show banner ads when the game starts.
ChooseBannerPosition	Choose where the banner ad should be displayed.
UseAdaptiveBannerSize	

	Enable support for adaptive banners based on screen orientation.
UseCustomAdaptiveBannerWidth	Enable to manually set the width for adaptive banners.
CustomAdaptiveBannerWidth	Specify a custom width for adaptive banners (only used if UseCustomAdaptiveBannerWidth is true).
AdaptiveBannerAdSizeOptions	Select which screen orientations to support for adaptive banners.
BannerAdSize	Set the standard banner ad size if adaptive is not used.
EnableTimedInterstitialAds	Enable to automatically show interstitial ads after a time interval.
InterstitialAdIntervalSeconds	Time interval (in seconds) to wait before showing an interstitial ad again.
ResetInterstitialAdTimerOnRewardedAd	Reset interstitial ad timer after showing a interstitial ad.
AppOpensToCheckBeforeShowingAppOpenAd	Number of app opens to check before showing an AppOpen Ad.
DelayShowAppOpenAd	Delay in seconds before showing an AppOpen Ad after the game starts.

Admob Ads Manager



Fields	Info
DebugAdInspector	Enable or disable the debugging of Ad Inspector.
AdInspectorButton	Button to open Ad Inspector for debugging.
ShowBannerAdButton	Button to show the banner ad.
ImageToUseToDisplayNativeAd	Image used to display the native ad.
ShowRewardedInterstitial	If true, shows a rewarded interstitial ad.
RewardedButton	Button associated with the rewarded ad.
script	Script attached to the button that will call the selected function.

scriptName	Name of the script associated with the button.
functionNames	List of function names to call from the script.
selectedFunctionIndex	Index of the selected function from the list.
ActionButtonsToInvokeInterstitialAds	Buttons that trigger the interstitial ads.
rewardedButtons	List of buttons that will trigger rewarded ads.

Admob Ads Mediation Part-2

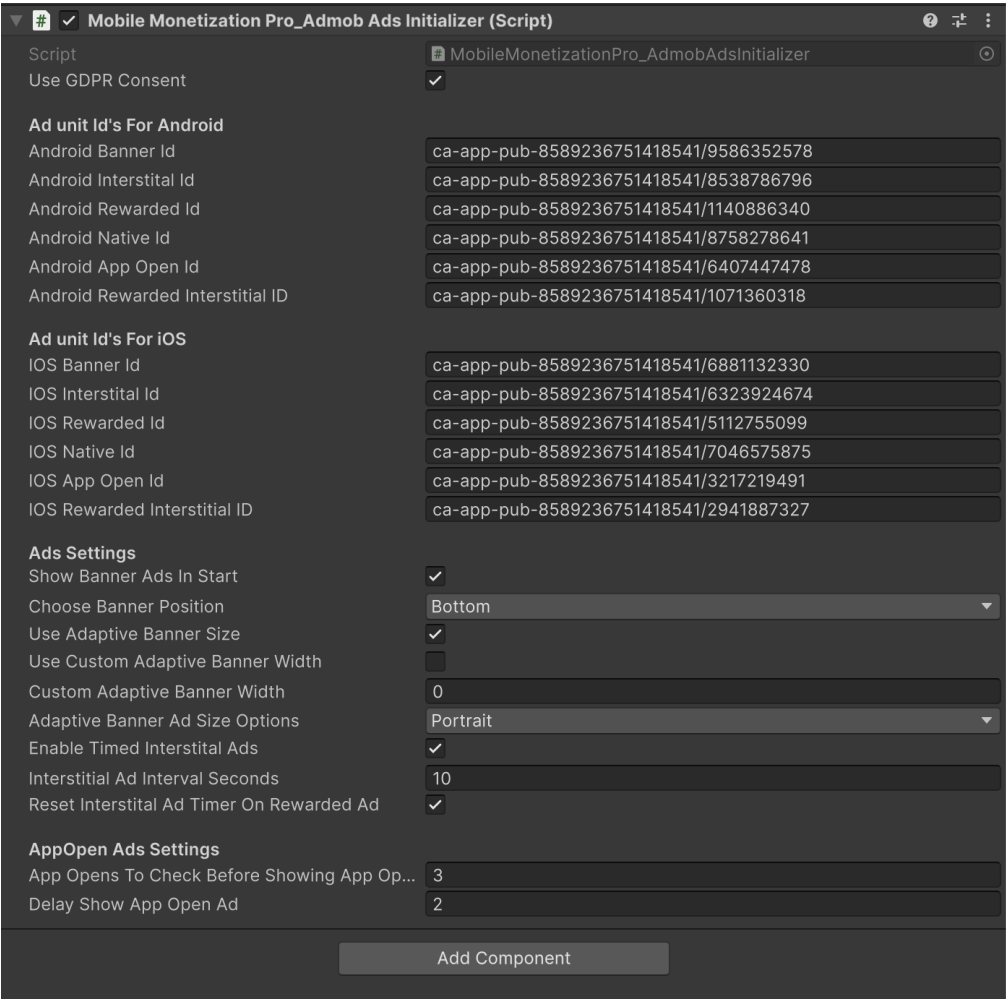
Introduction

In this video, we will be integrating **Admob Ads Mediation** into your mobile game using **Mobile Monetization Pro**.

You'll learn how to:

- Set up your Admob account for Mediation.
- Configure Admob SDK in Unity.
- Integrate and test ads.

Admob Ads_INITIALIZER



Script: MobileMonetizationPro_AdmobAdsInitializer

Use GDPR Consent: ☒

Ad unit Id's For Android

Android Banner Id	ca-app-pub-8589236751418541/9586352578
Android Interstitial Id	ca-app-pub-8589236751418541/8538786796
Android Rewarded Id	ca-app-pub-8589236751418541/1140886340
Android Native Id	ca-app-pub-8589236751418541/8758278641
Android App Open Id	ca-app-pub-8589236751418541/6407447478
Android Rewarded Interstitial ID	ca-app-pub-8589236751418541/1071360318

Ad unit Id's For iOS

IOS Banner Id	ca-app-pub-8589236751418541/6881132330
IOS Interstitial Id	ca-app-pub-8589236751418541/6323924674
IOS Rewarded Id	ca-app-pub-8589236751418541/5112755099
IOS Native Id	ca-app-pub-8589236751418541/7046575875
IOS App Open Id	ca-app-pub-8589236751418541/3217219491
IOS Rewarded Interstitial ID	ca-app-pub-8589236751418541/2941887327

Ads Settings

Show Banner Ads In Start	☒
Choose Banner Position	Bottom
Use Adaptive Banner Size	☒
Use Custom Adaptive Banner Width	☐
Custom Adaptive Banner Width	0
Adaptive Banner Ad Size Options	Portrait
Enable Timed Interstitial Ads	☒
Interstitial Ad Interval Seconds	10
Reset Interstitial Ad Timer On Rewarded Ad	☒

AppOpen Ads Settings

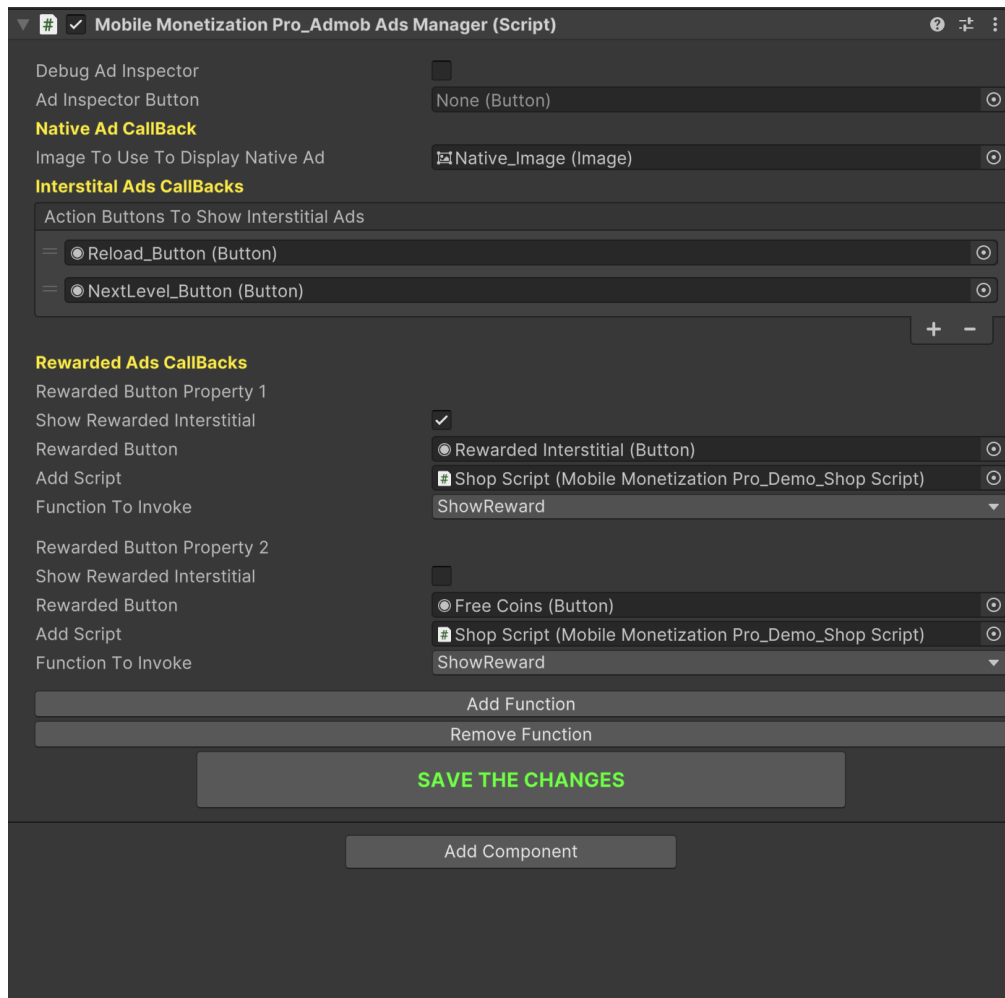
App Opens To Check Before Showing App Op...	3
Delay Show App Open Ad	2

Add Component

Fields	Info
UseGDPRConsent	Enable or disable GDPR consent support.
AndroidBannerId	Android AdMob Banner Ad Unit ID.
AndroidInterstitialId	Android AdMob Interstitial Ad Unit ID.
AndroidRewardedId	Android AdMob Rewarded Video Ad Unit ID.
AndroidNativeId	Android AdMob Native Ad Unit ID.
AndroidAppOpenId	Android AdMob App Open Ad Unit ID.
AndroidRewardedInterstitialID	Android AdMob Rewarded Interstitial Ad Unit ID.
IOSBannerId	iOS AdMob Banner Ad Unit ID.
IOSInterstitialId	iOS AdMob Interstitial Ad Unit ID.
IOSRewardedId	iOS AdMob Rewarded Video Ad Unit ID.
IOSNativeId	iOS AdMob Native Ad Unit ID.
IOSAppOpenId	iOS AdMob App Open Ad Unit ID.
IOSRewardedInterstitialID	iOS AdMob Rewarded Interstitial Ad Unit ID.
ShowBannerAdsInStart	Automatically show banner ads when the game starts.
ChooseBannerPosition	Choose where the banner ad should be displayed.
UseAdaptiveBannerSize	

	Enable support for adaptive banners based on screen orientation.
UseCustomAdaptiveBannerWidth	Enable to manually set the width for adaptive banners.
CustomAdaptiveBannerWidth	Specify a custom width for adaptive banners (only used if UseCustomAdaptiveBannerWidth is true).
AdaptiveBannerAdSizeOptions	Select which screen orientations to support for adaptive banners.
BannerAdSize	Set the standard banner ad size if adaptive is not used.
EnableTimedInterstitialAds	Enable to automatically show interstitial ads after a time interval.
InterstitialAdIntervalSeconds	Time interval (in seconds) to wait before showing an interstitial ad again.
ResetInterstitialAdTimerOnRewardedAd	Reset interstitial ad timer after showing a interstitial ad.
AppOpensToCheckBeforeShowingAppOpenAd	Number of app opens to check before showing an AppOpen Ad.
DelayShowAppOpenAd	Delay in seconds before showing an AppOpen Ad after the game starts.

Admob Ads Manager



Fields	Info
DebugAdInspector	Enable or disable the debugging of Ad Inspector.
AdInspectorButton	Button to open Ad Inspector for debugging.
ShowBannerAdButton	Button to show the banner ad.
ImageToUseToDisplayNativeAd	Image used to display the native ad.
ShowRewardedInterstitial	If true, shows a rewarded interstitial ad.
RewardedButton	Button associated with the rewarded ad.
script	Script attached to the button that will call the selected function.

scriptName	Name of the script associated with the button.
functionNames	List of function names to call from the script.
selectedFunctionIndex	Index of the selected function from the list.
ActionButtonsToInvokeInterstitialAds	Buttons that trigger the interstitial ads.
rewardedButtons	List of buttons that will trigger rewarded ads.

Admob Ads Callbacks Part-3

Introduction

In this video, we will learn how to set up **AdMob Ads Callbacks** in your mobile game using **Mobile Monetization Pro**.

Admob Ads - Banner & Interstitial Callback Event Example

Mobile Monetization Pro_Admob Ads Initializer (Script)

Script: MobileMonetizationPro_AdmobAdsInitializer

Use GDPR Consent: ☒

Ad unit Id's For Android

Android Banner Id	ca-app-pub-8589236751418541/9586352578
Android Interstitial Id	ca-app-pub-8589236751418541/8538786796
Android Rewarded Id	ca-app-pub-8589236751418541/1140886340
Android Native Id	ca-app-pub-8589236751418541/8758278641
Android App Open Id	ca-app-pub-8589236751418541/6407447478
Android Rewarded Interstitial ID	ca-app-pub-8589236751418541/1071360318

Ad unit Id's For iOS

iOS Banner Id	ca-app-pub-8589236751418541/6881132330
iOS Interstitial Id	ca-app-pub-8589236751418541/6323924674
iOS Rewarded Id	ca-app-pub-8589236751418541/5112755099
iOS Native Id	ca-app-pub-8589236751418541/7046575875
iOS App Open Id	ca-app-pub-8589236751418541/3217219491
iOS Rewarded Interstitial ID	ca-app-pub-8589236751418541/2941887327

Ads Settings

Show Banner Ads In Start	☒
Choose Banner Position	Bottom
Use Adaptive Banner Size	☒
Use Custom Adaptive Banner Width	☐
Custom Adaptive Banner Width	0
Adaptive Banner Ad Size Options	Portrait
Enable Timed Interstitial Ads	☒
Interstitial Ad Interval Seconds	10
Reset Interstitial Ad Timer On Rewarded Ad	☒

AppOpen Ads Settings

App Opens To Check Before Showing App Op...	3
Delay Show App Open Ad	2

Add Component

```
```csharp using System.Collections; using UnityEngine;
```

```
public class AdmobAdEvents : MonoBehaviour { private
MobileMonetizationPro.MobileMonetizationPro_AdmobAdsInitializer admobInitializer;
```

```
private void Start()
{
```

```
 admobInitializer = MobileMonetizationPro.MobileMonetizationPro_AdmobAd
```

```

 if (admobInitializer != null)
 {
 StartCoroutine(ShowInterstitialAutomatically());
 }
 else
 {
 Debug.LogWarning("AdmobAdsInitializer instance not found!");
 }
 }

 private void OnTriggerEnter(Collider other)
 {
 if (other.CompareTag("Player") && admobInitializer != null)
 {
 admobInitializer.HideBanner();
 }
 }

 private void OnTriggerExit(Collider other)
 {
 if (other.CompareTag("Player") && admobInitializer != null)
 {
 admobInitializer.ShowBanner();
 }
 }

 private IEnumerator ShowInterstitialAutomatically()
 {
 yield return new WaitForSeconds(5f);
 if (admobInitializer != null)
 {
 admobInitializer.ShowInterstitialAd(true);
 admobInitializer.ResetInterstitialAdTimer();
 }
 }
}'''

```

## Admob Ads - Rewarded Callback Event Example

```
``csharp using UnityEngine; using UnityEngine.SceneManagement; using
UnityEngine.UI;
```

```
public class ProgressionBar : MonoBehaviour { public Image fillImage; // Image
component to fill public float fillSpeed = 0.1f; // Speed at which the fill completes
```

```
public Button LoadLevelButton;
```

```
private void Start()
```

```
{
```

```
 if (fillImage == null)
```

```
 {
```

```
 Debug.LogError("Fill Image is not assigned!");
```

```
 return; // Exit if no image assigned
```

```
 }
```

```
 fillImage.fillAmount = 0f; // Ensure starting from 0
```

```
 StartFilling(); // Start filling normally if assigned
```

```
}
```

```
private void Update()
```

```
{
```

```
 if(MobileMonetizationPro.MobileMonetizationPro_AdmobAdsInitializer.ins
```

```
 {
```

```
 // give reward
```

```
 LoadLevelButton.gameObject.SetActive(true);
```

```
 MobileMonetizationPro.MobileMonetizationPro_AdmobAdsInitializer.ins
```

```
 }
```

```
}
```

```
public void StartFilling()
```

```
{
```

```
 StartCoroutine(FillProgressionBar());
```

```
}
```

```
private System.Collections.IEnumerator FillProgressionBar()
```

```
{
```

```
 while (fillImage.fillAmount < 1f)
```

```
{
 fillImage.fillAmount += fillSpeed * Time.deltaTime; // Increment
 fillImage.fillAmount = Mathf.Clamp01(fillImage.fillAmount); // Cla
 yield return null; // Wait for next frame
}

// Fill completed
Debug.Log("Fill Completed!");

 if (MobileMonetizationPro.MobileMonetizationPro_AdmobAdsInitializer.i
 MobileMonetizationPro.MobileMonetizationPro_AdmobAdsInitializer.ins
}

}

...
```

# LevelPlay Ads Setup Part-1

## Introduction

In this video, you'll learn how to integrate **LevelPlay** ads into your Unity mobile game.

We will cover the following key concepts:

- **Setting up LevelPlay:** Learn how to create an account and set up your project in LevelPlay.
- **Integrating LevelPlay SDK:** A step-by-step guide on adding the LevelPlay SDK to your Unity project.
- **Ad formats:** Understand the different types of ads available (banner ads, interstitials, rewarded videos) and how to implement each one.
- **Testing on android and iOS:** We will test the game on both Android and iOS device.

By the end of the video, you'll have a fully functional ad setup in your game.

## LevelPlay Ads Initializer

Script: MobileMonetizationPro\_LevelPlayInitializer

Use GDPR Consent: ☐

**App Key's**

Android App Key	21bb0d8f5
Android Banner Ad Unit ID	707z1z27k7a533i
Android Interstitial Ad Unit Id	v5r0azvyzt5tb8ya
Android Rewarded Ad Unit Id	1j6hdsbfk9tie656
iOS App Key	21bc75215
iOS Banner Ad Unit ID	urqyo76pbwysjc0
iOS Interstitial Ad Unit Id	3qkt9xejy731sejw
iOS Rewarded Ad Unit Id	mq7iwef30on0n0i2

**Ad Settings**

Show Banner Ads In Start	<input checked="" type="checkbox"/>
Enable Timed Interstitial Ads	<input checked="" type="checkbox"/>
Interstitial Ad Interval Seconds	10
Reset Interstitial Ad Timer On Rewarded Ad	<input checked="" type="checkbox"/>

Open Ad Network Dashboard

Open Ad Network Integration Manager

Configure Ad Network Mediation Settings

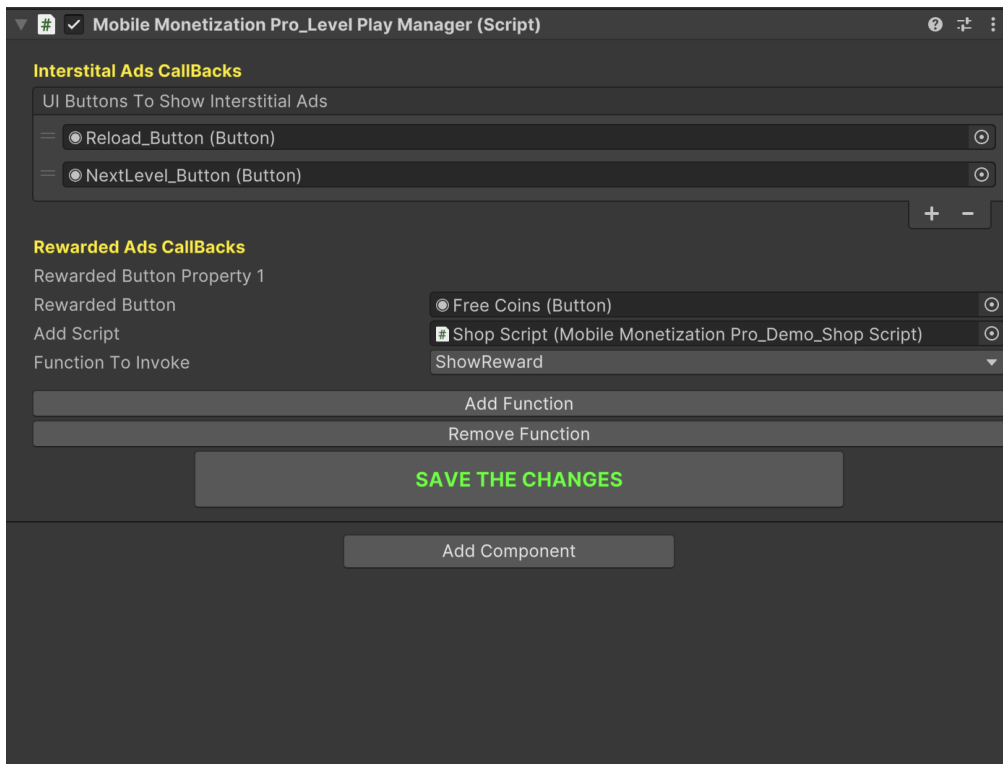
Configure Ad Network Mediated Network Settings

Add Component

Fields	Info
--------	------

UseGDPRConsent	Flag to indicate whether to use GDPR consent before loading ads.
AndroidAppKey	The app key for the Android platform.
AndroidBannerAdUnitID	The ad unit ID for displaying banner ads on Android.
AndroidInterstitialAdUnitId	The ad unit ID for displaying interstitial ads on Android.
AndroidRewardedAdUnitId	The ad unit ID for displaying rewarded ads on Android.
iOSAppKey	The app key for the iOS platform.
iOSBannerAdUnitID	The ad unit ID for displaying banner ads on iOS.
iOSInterstitialAdUnitId	The ad unit ID for displaying interstitial ads on iOS.
iOSRewardedAdUnitId	The ad unit ID for displaying rewarded ads on iOS.
ShowBannerAdsInStart	Flag to indicate whether to show banner ads at the start of the game.
EnableTimedInterstitialAds	Flag to indicate whether interstitial ads should be shown after a time interval.
InterstitialAdIntervalSeconds	The interval duration in seconds to wait before showing a timed interstitial ad.
ResetInterstitAdTimerOnRewardedAd	Flag to indicate whether the interstitial ad timer should be reset when a rewarded ad is shown.

# LevelPlay Ads Manager



Fields	Info
ShowBannerAdButton	Button to show banner ad when clicked.
RewardedButton	Button to show rewarded video when clicked.
script	Script that contains the functions to be invoked.
scriptName	Name of the script associated with the functions.
functionNames	List of function names that can be invoked from the script.
selectedFunctionIndex	Index of the currently selected function to invoke.
UIButtonsToShowInterstitialAds	List of buttons that show interstitial ads when clicked.
rewardedButtons	List of rewarded buttons that trigger rewarded video ads.

# LevelPlay Ads Callbacks Part-2

## Introduction

In this video, we will learn how to set up **LevelPlay Ads Callbacks** in your mobile game using **Mobile Monetization Pro**.

## LevelPlay Ads - Banner & Interstitial Callback Event Example

```
``csharp using System.Collections; using UnityEngine;

public class LevelPlayAdEventsExample : MonoBehaviour { private
MobileMonetizationPro.MobileMonetizationPro_LevelPlayInitializer LevelPlayInitializer;

private void Start()
{
 LevelPlayInitializer = MobileMonetizationPro.MobileMonetizationPro_LevelPlayInitializer;
 StartCoroutine>ShowInterstitialAutomatically());
}

private void OnTriggerEnter(Collider other)
{
 if (other.CompareTag("Player") && LevelPlayInitializer != null)
 {
 LevelPlayInitializer.HideBannerAd();
 }
}

private void OnTriggerExit(Collider other)
{
 if (other.CompareTag("Player") && LevelPlayInitializer != null)
 {
 LevelPlayInitializer.ShowBannerAd();
 }
}

private IEnumerator ShowInterstitialAutomatically()
{
```

```

 yield return new WaitForSeconds(10f);
 if (LevelPlayInitializer != null)
 {
 LevelPlayInitializer.ShowInterstitial(true);
 LevelPlayInitializer.ResetInterstitialAdTimer();
 }
 }
}'''

```

## LevelPlay Ads - Rewarded Callback Event Example

```

'''csharp using UnityEngine; using UnityEngine.SceneManagement; using
UnityEngine.UI;

```

```

public class ProgressionBar : MonoBehaviour { public Image fillImage; // Image
component to fill public float fillSpeed = 0.1f; // Speed at which the fill completes

```

```

public Button LoadLevelButton;

```

```

private void Start()
{

```

```

 if (fillImage == null)
 {
 Debug.LogError("Fill Image is not assigned!");
 return; // Exit if no image assigned
 }

```

```

 fillImage.fillAmount = 0f; // Ensure starting from 0
 StartFilling(); // Start filling normally if assigned
}

```

```

private void Update()
{

```

```

 if (MobileMonetizationPro.MobileMonetizationPro_LevelPlayInitializer.i
 {
 // give reward
 LoadLevelButton.gameObject.SetActive(true);
 MobileMonetizationPro.MobileMonetizationPro_LevelPlayInitializer.i

```

```

 }
}
public void StartFilling()
{
 StartCoroutine(FillProgressionBar());
}

private System.Collections.IEnumerator FillProgressionBar()
{
 while (fillImage.fillAmount < 1f)
 {
 fillImage.fillAmount += fillSpeed * Time.deltaTime; // Increment
 fillImage.fillAmount = Mathf.Clamp01(fillImage.fillAmount); // Cla
 yield return null; // Wait for next frame
 }

 // Fill completed
 Debug.Log("Fill Completed!");

 if (MobileMonetizationPro.MobileMonetizationPro_LevelPlayInitializer.i
 MobileMonetizationPro.MobileMonetizationPro_LevelPlayInitializer.i
 }
}'''

```

# Applovin Ads Part-1

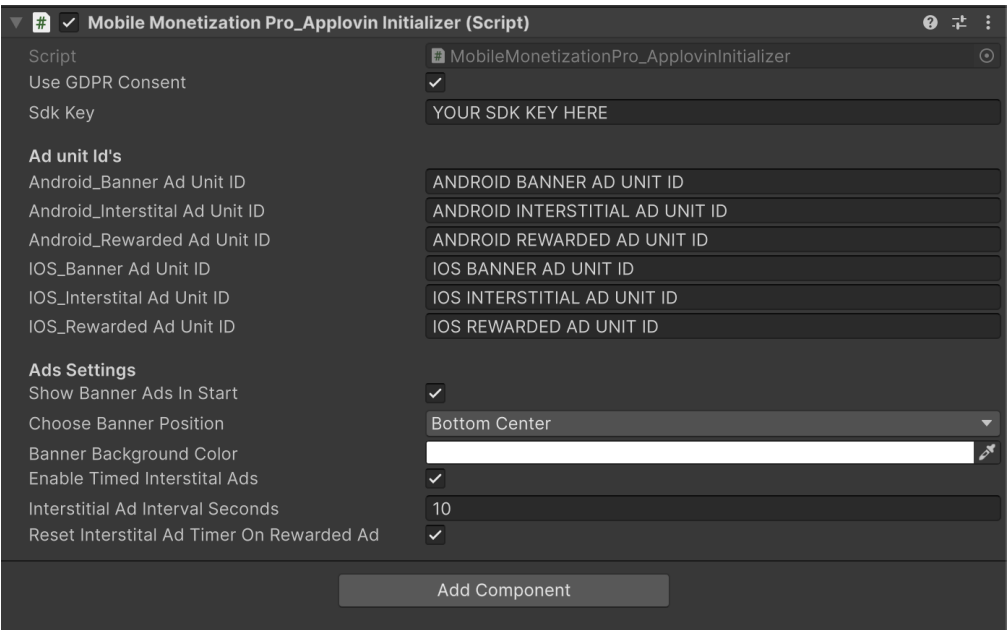
## Introduction

In this video, we take the first step in integrating **Applovin Ads** into your mobile game using **Mobile Monetization Pro**.

You'll learn how to:

- Set up your Applovin account.
- Configure Applovin SDK in Unity.
- Integrate and test applovin ads.

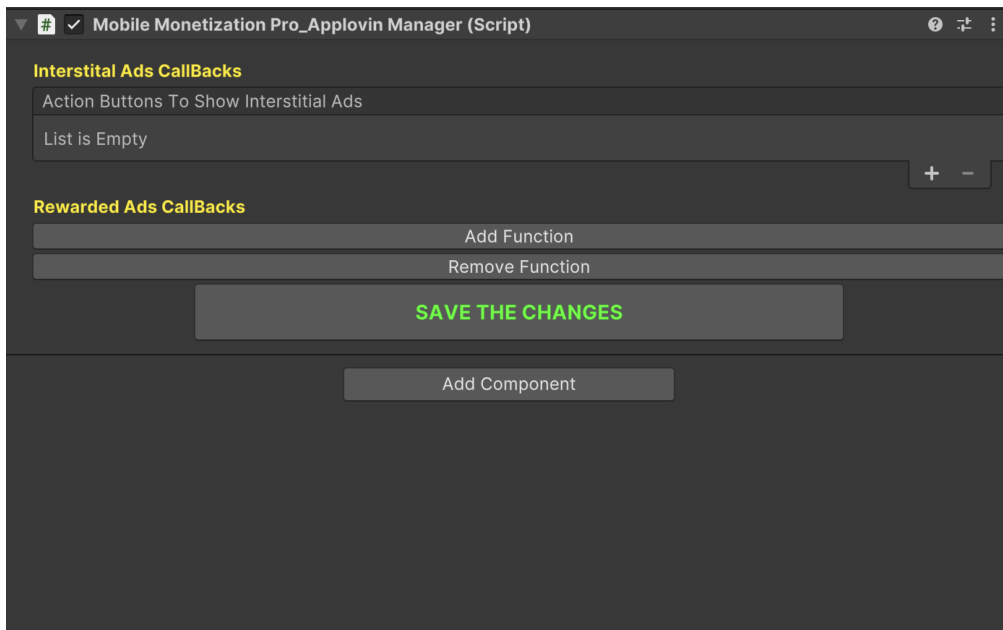
## Applovin Ads Initializer



Fields	Info
UseGDPRConsent	If enabled, the SDK will wait for GDPR consent before initializing ads.
SdkKey	Your AppLovin MAX SDK Key. Required to initialize the SDK.

Android_BannerAdUnitID	Banner Ad Unit ID for Android.
Android_InterstitialAdUnitID	Interstitial Ad Unit ID for Android.
Android_RewardedAdUnitID	Rewarded Ad Unit ID for Android.
IOS_BannerAdUnitID	Banner Ad Unit ID for iOS.
IOS_InterstitialAdUnitID	Interstitial Ad Unit ID for iOS.
IOS_RewardedAdUnitID	Rewarded Ad Unit ID for iOS.
ShowBannerAdsInStart	If enabled, banner ads will be shown automatically at the start.
ChooseBannerPosition	Position on the screen where the banner ad will be displayed.
BannerBackgroundColor	Background color for the banner ad.
EnableTimedInterstitialAds	If enabled, interstitial ads will be shown automatically after a timed interval.
InterstitialAdIntervalSeconds	Time interval (in seconds) after which an interstitial ad will be shown.
ResetInterstitialAdTimerOnRewardedAd	If enabled, the interstitial ad timer will reset after showing a rewarded ad.

# Applovin Ads Manager



Fields	Info
ShowBannerAdButton	Button that shows the banner ad when clicked.
FunctionInfo.RewardedButton	Button that triggers a rewarded ad and invokes a function when the ad is completed.
FunctionInfo.script	Script containing the function to invoke after ad completion.
FunctionInfo.scriptName	Name of the selected script.
FunctionInfo.functionNames	List of function names available in the selected script.
FunctionInfo.selectedFunctionIndex	Index of the function selected to be called from the list.
ActionButtonsToInvokeInterstitialAds	Buttons that trigger an interstitial ad when clicked.
rewardedButtons	Auto-generated list of rewarded buttons associated with functions.

# Applovin Ads Mediation Part-2

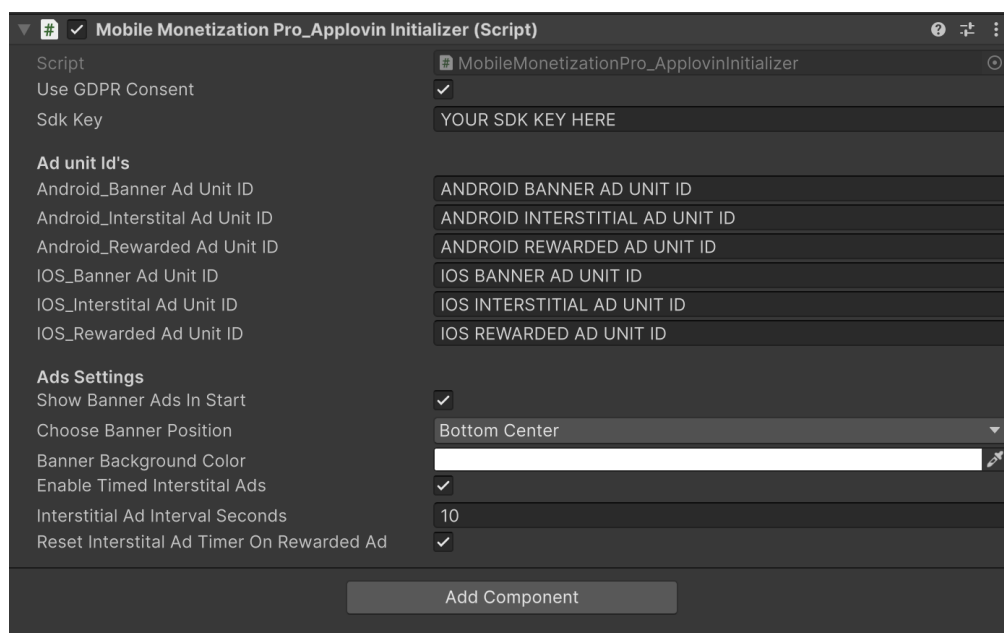
## Introduction

In this video, we will be integrating **Applovin Ads Mediation** into your mobile game using **Mobile Monetization Pro**.

You'll learn how to:

- Set up your Applovin account for Mediation.
- Configure Applovin SDK in Unity.
- Integrate and test ads.

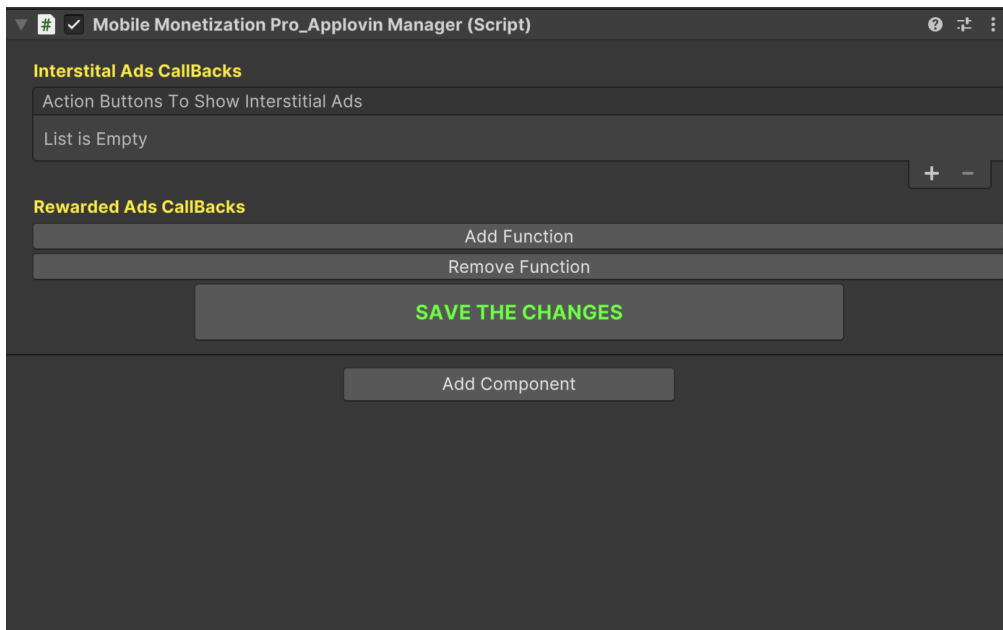
## Applovin Ads Initializer



Fields	Info
UseGDPRConsent	If enabled, the SDK will wait for GDPR consent before initializing ads.
SdkKey	Your AppLovin MAX SDK Key. Required to initialize the SDK.

Android_BannerAdUnitID	Banner Ad Unit ID for Android.
Android_InterstitialAdUnitID	Interstitial Ad Unit ID for Android.
Android_RewardedAdUnitID	Rewarded Ad Unit ID for Android.
IOS_BannerAdUnitID	Banner Ad Unit ID for iOS.
IOS_InterstitialAdUnitID	Interstitial Ad Unit ID for iOS.
IOS_RewardedAdUnitID	Rewarded Ad Unit ID for iOS.
ShowBannerAdsInStart	If enabled, banner ads will be shown automatically at the start.
ChooseBannerPosition	Position on the screen where the banner ad will be displayed.
BannerBackgroundColor	Background color for the banner ad.
EnableTimedInterstitialAds	If enabled, interstitial ads will be shown automatically after a timed interval.
InterstitialAdIntervalSeconds	Time interval (in seconds) after which an interstitial ad will be shown.
ResetInterstitialAdTimerOnRewardedAd	If enabled, the interstitial ad timer will reset after showing a rewarded ad.

## Applovin Ads Manager



Fields	Info
ShowBannerAdButton	Button that shows the banner ad when clicked.
FunctionInfo.RewardedButton	Button that triggers a rewarded ad and invokes a function when the ad is completed.
FunctionInfo.script	Script containing the function to invoke after ad completion.
FunctionInfo.scriptName	Name of the selected script.
FunctionInfo.functionNames	List of function names available in the selected script.
FunctionInfo.selectedFunctionIndex	Index of the function selected to be called from the list.
ActionButtonsToInvokeInterstitialAds	Buttons that trigger an interstitial ad when clicked.
rewardedButtons	Auto-generated list of rewarded buttons associated with functions.

# LiftOff Ads

## Introduction

In this video, we take the first step in integrating **LiftOff Ads(Vungle Ads)** into your mobile game using **Mobile Monetization Pro**.

You'll learn how to:

- Set up your LiftOff ads account.
- Configure LiftOff SDK in Unity.
- Integrate and test vunLiftOffgle ads.

## LiftOff Ads Initializer

#

✓

Mobile Monetization Pro\_Lift Off Ads Initializer (Script)

?

↺

⋮

Script

Use GDPR Consent

APP ID HERE

IOS App ID

Android App ID

PLACEMENT ID HERE

Android Interstitial Placement ID

Android Banner Placement ID

Android Rewarded Placement ID

IOS Interstitial Placement ID

IOS Banner Placement ID

IOS Rewarded Placement ID

AD SETTINGS

Show Banner Ads In Start

Banner Size

Banner Position

Enable Timed Interstitial Ads

Interstitial Ad Interval Seconds

Reset Interstitial Ad Timer On Rewarded Ad

Delaying Ads load time so the sdk can be initialized.

Delay Ad Loading Time

After loading banner add some delay before showing it.

Delay Banner Show Time

Add Component

MobileMonetizationPro\_LiftOffAdsInitializer

✓

ios\_app\_id

android\_app\_id

✓

Vungle Ad Size Banner

Bottom Center

✓

10

✓

2

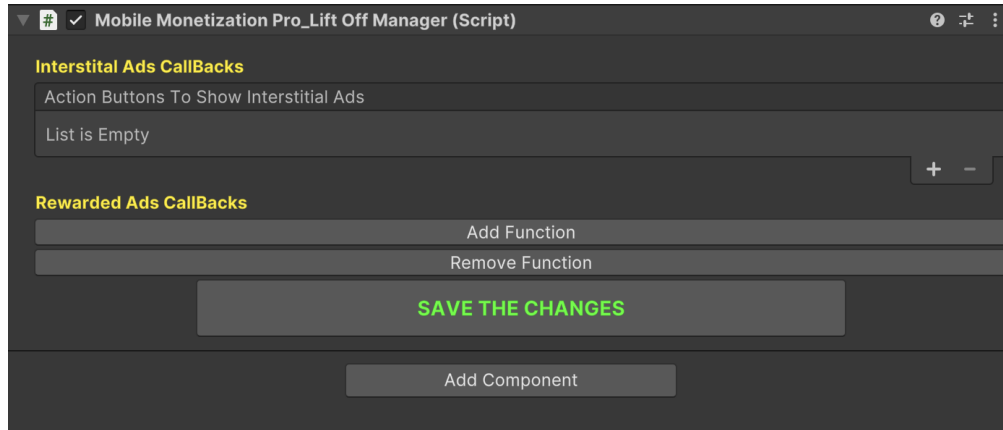
2

Fields	Info
--------	------

UseGDPRConsent	Enable or disable GDPR consent prompt before initializing ads.
IOSAppID	Your iOS App ID provided by LiftOff (Vungle) dashboard.
AndroidAppID	Your Android App ID provided by LiftOff (Vungle) dashboard.
AndroidInterstitialPlacementID	Placement ID for Interstitial Ads on Android.
AndroidBannerPlacementID	Placement ID for Banner Ads on Android.
AndroidRewardedPlacementID	Placement ID for Rewarded Ads on Android.
IOSInterstitialPlacementID	Placement ID for Interstitial Ads on iOS.
IOSBannerPlacementID	Placement ID for Banner Ads on iOS.
IOSRewardedPlacementID	Placement ID for Rewarded Ads on iOS.
ShowBannerAdsInStart	Automatically show banner ads when the app starts.
BannerSize	Select the banner size for your ads.
BannerPosition	Select the banner position on the screen.
EnableTimedInterstitialAds	Enable automatic timed Interstitial ads after certain intervals.
InterstitialAdIntervalSeconds	Time interval (in seconds) between each Interstitial ad.
ResetInterstitialAdTimerOnRewardedAd	Reset Interstitial ad timer when a Rewarded ad is watched.
DelayAdLoadingTime	Delay time (in seconds) before loading ads after initialization.
DelayBannerShowTime	

Delay time (in seconds) after loading a banner ad before showing it.

## LiftOff Ads Manager



Fields	Info
ShowBannerAdButton	Button that, when clicked, manually shows the banner ad.
RewardedButton	Button that triggers a rewarded ad and invokes a linked function after the ad is completed.
script	Script containing the function to invoke after the rewarded ad is completed.
scriptName	Name of the script associated with this function info (for reference).
functionNames	List of function names available in the assigned script.
selectedFunctionIndex	Index of the selected function from the available functions list.
ActionButtonsToInvokeInterstitialAds	Buttons that trigger interstitial ads when clicked.
rewardedButtons	List of rewarded buttons collected for setting up listeners.

functions	List storing reward button configurations and their corresponding function call information.
-----------	----------------------------------------------------------------------------------------------

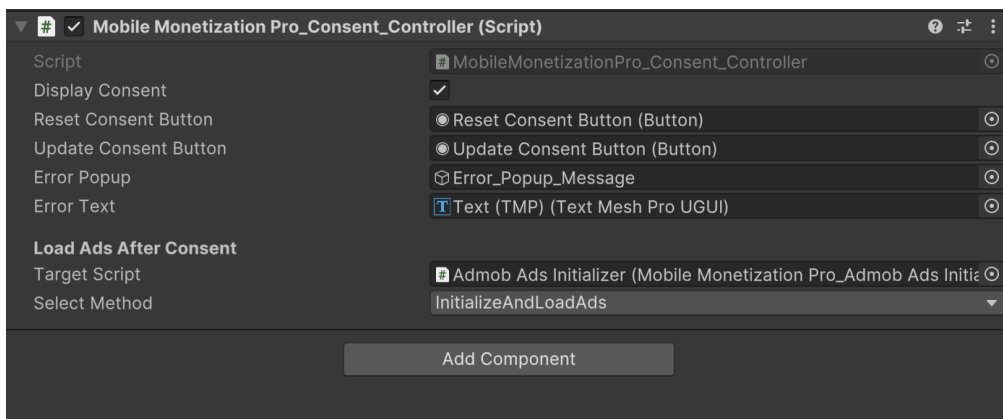
# GDPR Setup

## Introduction

In this video, you'll learn how to implement **GDPR consent message** in your mobile game using **Mobile Monetization Pro**.

We cover: - Display a GDPR popup for players in supported regions - Request consent for personalized and non-personalized ads

## Consent Controller Script



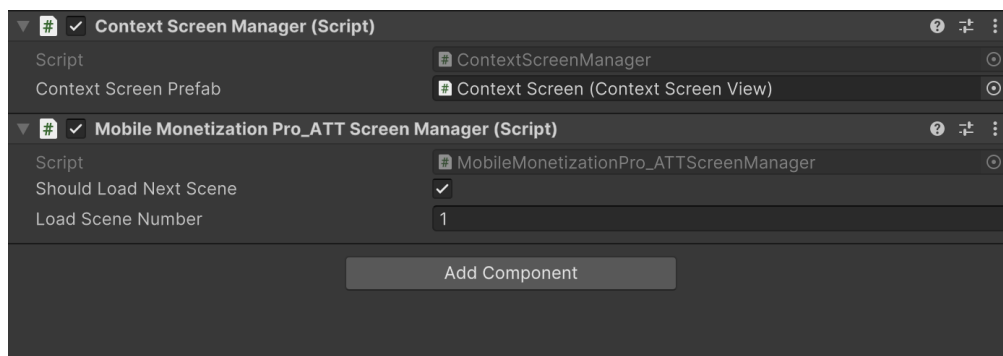
Fields	Info
DisplayConsent	Set to true to display the GDPR consent
ResetConsentButton	Button to reset consent.
UpdateConsentButton	Button to show user consent settings again.
ErrorPopup	GameObject with the error popup.
ErrorText	Error message for the error popup.
targetScript	Drag a MonoBehaviour from the scene with public void methods.
selectedMethodName	Name of the public void method to call.

# App Tracking Transparency Setup

## Introduction

The tutorial focuses on adding an **App Tracking Transparency (ATT) popup** for iOS applications. It is essential to ensure that the platform is set to iOS before proceeding with the integration steps.

## Setting Up the Environment



- Users are instructed to navigate to the Tools section and select Mobile Monetization Pro and Integration Tool.
- The integration option for the ATT popup is selected, followed by downloading the necessary package from the Unity registry.
- It is necessary to search for and install the iOS 14 advertising support package from package manager.

## Importing Required Assets

- After installation, users need to access the samples and import the context screen.

## Displaying the ATT Popup

- The first method to display the ATT popup involves showing it in the separate scene, specifically scene number zero.
- Users are guided to copy specific elements from the context screen sample and paste them into the Mobile Monetization Pro demo game scene.
- When the game starts, the ATT popup will appear, prompting the user to make a choice regarding tracking.

## **Building and Testing the Application**

- To test the implementation, a build for the iOS device must be created by navigating to the file build settings.
- Once the project is built, users should open the Unity iPhone XC workspace and connect their iPhone.
- After selecting the appropriate category, the game can be installed on the iOS device for testing.

## **User Interaction with the Popup**

- Upon launching the game, users will see the ATT prompt asking for permission to track their activity across other apps and websites.
- If the user selects 'allow' and restarts the game, the ATT popup will not reappear, thus confirming the user's choice.

## **Alternative Method for Displaying the ATT Popup**

- An alternative method involves deleting the context screen sample and utilizing the iOS 14 advertising support context screen.
- Users can then access the Mobile Monetization Pro Tools and drag the ATT screen manager script into the scene.
- It is important to add this scene to the build settings to ensure the ATT popup appears in the first scene of the game.

## **Combining ATT and GDPR Popups**

- If mobile ads are being used, it may also be necessary to implement a GDPR popup alongside the ATT popup.
- Users can specify that the ATT popup is shown in the first scene, while the GDPR popup can be displayed when the second scene loads.
- To facilitate this, users should check the option to load the next scene and provide the corresponding scene number in the build settings.

## **Final Testing and Conclusion**

- After configuring the scenes, users can create a build to test the functionality of the popups.
- When the first scene loads, the ATT popup will appear, and upon clicking 'next,' the second scene will load successfully without displaying the ATT popup again.

- The tutorial concludes with the successful implementation of the App Tracking Transparency popup in the game.

# Setup IAP on Google Play & App Store

## Introduction

In this video, you'll learn how to set up In-App Purchases (IAP) for your mobile game on Google Play Console and App Store Connect. This step-by-step tutorial walks you through the process of creating in-app products and configuring them correctly on both platforms.

### What's Covered

- Setting up in-app products on Google Play Console and App Store Connect
- Key settings and configurations required for successful IAP setup
- Connecting your in-app products with your Unity project

## Setting Up In-App Products on Google Play Console

To offer in-app purchases (IAP) on Google Play, you must configure your products in the Google Play Console.

- **Consumable Products:** Items like in-game currency or consumables that are used once and can be purchased multiple times. These need to be set up on Google Play.
- **Non-Consumable Products:** Items that users buy once and own permanently, such as skins, weapons, or other unlockable content. You will need to configure these for one-time purchases.
- **Subscriptions:** If you're offering subscriptions, such as recurring premium content or services, you'll need to configure subscription products with renewal periods (e.g., weekly, monthly, yearly).

Google Play Console allows you to set the pricing, manage localized prices.

## Setting Up In-App Products on App Store Connect

For iOS applications, Apple uses App Store Connect for managing IAP products.

- **Consumables:** Just like Google Play, consumable items on iOS are items that can be bought and used (like power-ups, in-game currency, etc.).
- **Non-Consumables:** These items are permanently available to the user after purchase, such as extra levels, skins, or game features.
- **Subscriptions:** If you're offering recurring services, such as a premium membership, these subscriptions are set up with renewal cycles and can provide users with ongoing access to content.

## Managing IAP Products

Both stores allow you to edit or update your IAP products after they are live. For example, you can:

- Add new in-app products or subscriptions.
- Modify prices or availability.
- Remove or discontinue old products.

This management is essential for keeping your game's monetization up to date and aligned with any seasonal changes or content updates.

## Testing Your IAP Products

Before going live with your in-app purchases, you need to test them thoroughly:

**Google Play Console:** You can create test accounts and simulate purchases, allowing you to verify that everything is working as expected before making it available to the public.

**App Store Connect:** Apple offers sandbox accounts for testing your in-app purchases. You can make purchases without actually being charged and test the full purchasing flow to ensure that everything is functioning properly.

# In App Purchase Setup

## Introduction

In this video, you'll learn how to implement **In App Purchase** in your mobile game using **Mobile Monetization Pro**.

## IAP Manager Script

**Mobile Monetization Pro\_IAP Manager (Script)**

Script: MobileMonetizationPro\_IAPManager

**Restore Purchases For iOS**

Restore Purchase Button: Restore purchase button (Button)

**Consumable Item 1**

UI Button: CHEST (Button)

Price Text: Price Text (Text Mesh Pro UGUI)

Can Buy In Quantity: ☒

Product ID: get5000coins

Add Script: Shop Script (Mobile Monetization Pro\_Demo\_Shop Script)

Select Function To Invoke: SetAndBuyMoreCoins

Parameter 1 (Int32): 5000

Parameter 2 (GameObject): Reward

**Non Consumable Item 1**

UI Button: No\_Ads Button (Button)

Price Text: Price Text (Text Mesh Pro UGUI)

Product ID: remove\_ads

Remove Ads On Purchase: ☒

Add Script: Shop Script (Mobile Monetization Pro\_Demo\_Shop Script)

Select Function To Invoke: BuyRemoveAdsFromGame

Call Function On Refund: ☒

Restart Ads On Refund: ☒

Add Script On Refund: Shop Script (Mobile Monetization Pro\_Demo\_Shop Script)

Select Function To Invoke: RestartAdsFromGame

**Subscription Item1**

UI Button: star (Button)

Price Text: Price Text (Text Mesh Pro UGUI)

Product ID: getvipbadge

Add Script: Shop Script (Mobile Monetization Pro\_Demo\_Shop Script)

Select Function To Invoke On Activation: ActivateWeeklySubscription

Select Function To Invoke On Deactivation: DeactivateWeeklySubscription

Remove Add Consumable Item

Remove Add Non-Consumable Item

Remove Add Subscription Item

**SAVE THE CHANGES**

Fields	Info
UIButton	The button for the consumable item.
PriceText	

	Text to display the price of the consumable item.
CanBuyInQuantity	If true, allows the item to be bought in a specific quantity.
ProductId	The product ID for the consumable item.
AddScript	The MonoBehaviour script to be added when the item is bought.
FunctionToInvoke	The function to invoke when the item is purchased.
FunctionParameters	A list of parameters to pass to the function when invoking it.
UIButton	The button for the non-consumable item.
PriceText	Text to display the price of the non-consumable item.
ProductId	The product ID for the non-consumable item.
RemoveAdsFromGame	If true, removes ads from the game when the non-consumable item is purchased.
AddScript	The MonoBehaviour script to be added when the item is bought.
FunctionToInvoke	The function to invoke when the item is purchased.
FunctionParameters	A list of parameters to pass to the function when invoking it.
CallFunctionOnRefund	If true, invokes the specified function when a refund occurs.

RestartAdsOnRefund	If true, restarts ads on refund.
AddScriptOnRefund	The MonoBehaviour script to be added on refund.
FunctionToInvokeOnRefund	The function to invoke on refund.
FunctionParametersForRefund	A list of parameters to pass to the function on refund.
UIButton	The button for the subscription item.
PriceText	Text to display the price of the subscription.
ProductId	The product ID for the subscription item.
AddScript	The MonoBehaviour script to be added when the subscription is purchased.
FunctionToInvokeForActivatingSubscription	The function to invoke for activating the subscription.
FunctionToInvokeForDeactivatingSubscription	The function to invoke for deactivating the subscription.
ActivationFunctionParameters	A list of parameters to pass to the activation function when activating the subscription.
DeactivationFunctionParameters	A list of parameters to pass to the deactivation function when deactivating the subscription.

# Local Notifications

## Introduction

The tutorial focuses on integrating **Mobile Notifications** into a Unity project, providing step-by-step instructions to achieve this.

## Installing Mobile Notifications

- Users are instructed to install mobile notifications through the Unity Package Manager.
- To find the mobile notifications package, users should search for 'mobile notifications' in the Unity Registry and click on 'Install' after selecting the appropriate package.
- Once installed, users should close the Package Manager and import the necessary scripts for the mobile notifications.

## Configuring Project Settings

- After importing the scripts, users need to navigate to the Project Settings and select the Mobile Notifications section.
- Icons for notifications must be added, starting with the small icon, which users can drag and drop from the Mobile Monetization Pro UI icon section.
- It is essential to ensure that the 'Read and Write' option is checked for the icons before applying the changes.
- Users are also guided to add a large icon by clicking the plus sign, choosing an image, and ensuring the read and write option is also selected for this icon.

## iOS Notification Setup

- For iOS setup, users must check the 'Request Authorization on App Launch' option and enable push notifications.

## Creating Notification Objects

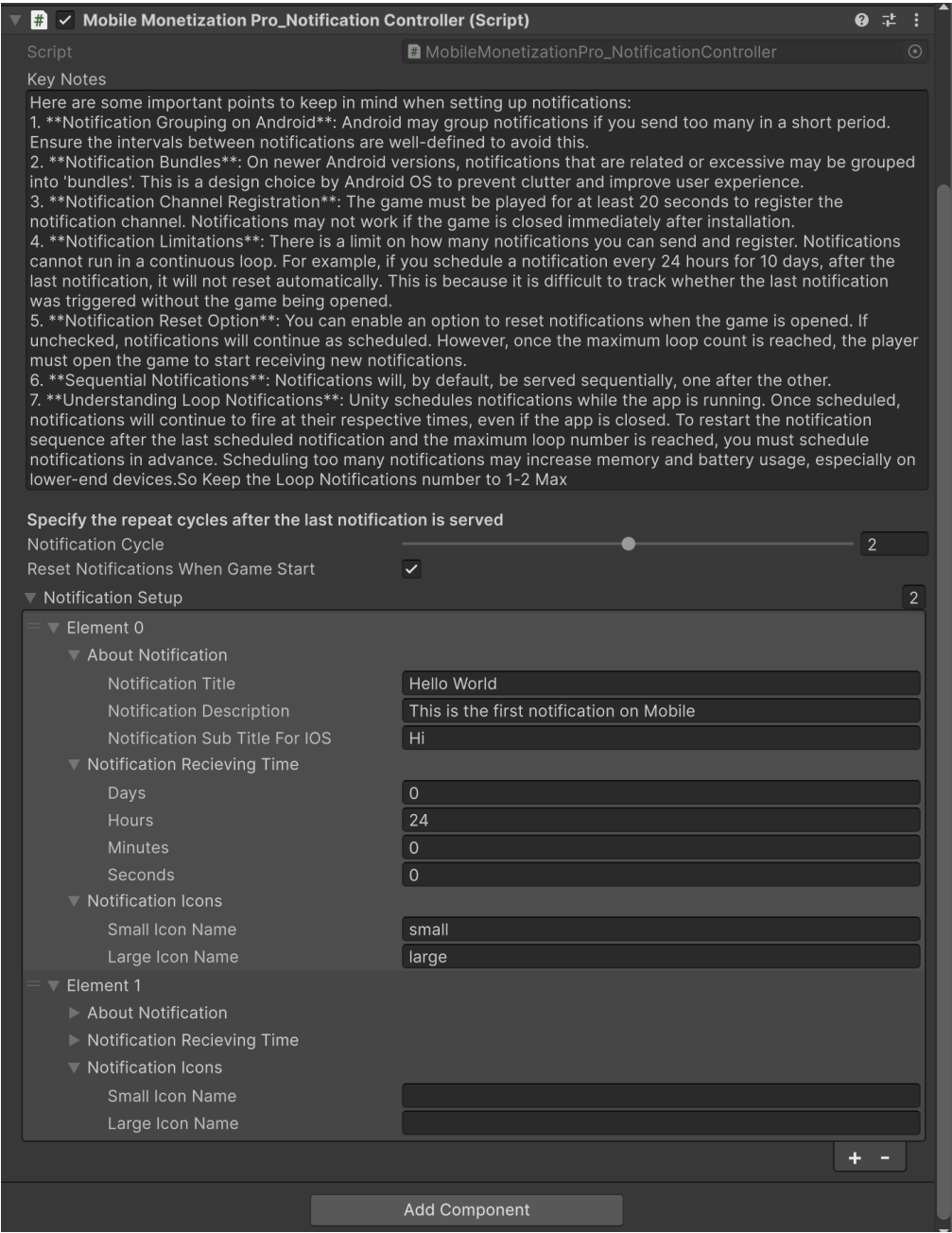
- Users are instructed to create an empty GameObject named 'Mobile Notifications' within the hierarchy.

- They should then navigate to the folder - Mobile Monetization Pro < Tools < Mobile Notification Manager and drag the relevant script onto this GameObject.
- In this section, users can set the notification title, description, and subtitle, with examples provided for clarity.
- The tutorial emphasizes selecting a notification receiving time, which can be set in seconds, minutes, hours, or days.

## **Finalizing Notification Setup**

- Users are reminded to input the same icon names used in project settings into the notification setup to ensure consistency.
- Once all settings are configured, the mobile notification setup is complete, allowing users to proceed to build their project for Android.

# Local Notifications Script



Fields	Info
DebugTextForiOS	Assign a TextMeshProUGUI component here to display debug messages specifically for iOS notification behavior during development. (Optional)
NotificationCycle	Specifies how many times the full notification schedule should loop after completing once.

	Example: If set to 2, the notifications will repeat twice after the first full cycle.
ResetNotificationsWhenGameStart	Enable this to reset and re-schedule notifications every time the player starts the game. Disable to continue the original notification schedule without resetting.
NotificationSetup	List of notifications to schedule. Each entry defines its title, description, delay time, and icons.
AboutNotification	Define the title, body text, and optional subtitle (for iOS) for this notification.
NotificationReceivingTime	Set the delay after which this notification should appear. You can specify days, hours, minutes, and seconds.
NotificationIcons	Assign custom small and large icons for this notification on Android.
NotificationTitle	The title text displayed on the notification.
NotificationDescription	The main body text of the notification.
NotificationSubTitleForIOS	A subtitle for the notification (used only on iOS devices).
Days	Delay in days before this notification is triggered.
Hours	Delay in hours before this notification is triggered.
Minutes	Delay in minutes before this notification is triggered.
Seconds	Delay in seconds before this notification is triggered.
SmallIconName	The name of the small icon file (must exist in the Android project Resources).

LargelconName	The name of the large icon file (must exist in the Android project Resources).
---------------	--------------------------------------------------------------------------------

### 🔑 Key Notes:

- 📱 **Notification Grouping on Android:** Android may group notifications if you send too many in a short period. Make sure to space them out properly.
  - 📦 **Notification Bundles:** Newer Android versions may group related notifications into "bundles" to improve the user experience.
  - ⌚ **Notification Channel Registration:** The player must play the game for at least 20 seconds after installation to properly register notification channels.
  - 🚫 **Notification Limitations:** Notifications cannot run indefinitely. Once the last scheduled notification fires, they will not restart automatically unless rescheduled.
  - 🔄 **Notification Reset Option:** Enable reset to start the notification cycle again when the player opens the game. Otherwise, notifications continue until the set cycle limit is reached.
  - 🔄 **Sequential Notifications:** Notifications are sent one after the other based on the scheduled time intervals.
  - 🛠️ **Understanding Loop Notifications:** Unity schedules notifications when the app is running. Even if the app is closed, scheduled notifications will fire. However, scheduling too many notifications may consume more memory and battery — keep loop counts low (preferably 1–2) for best performance.
-

# Remote Notifications

## Introduction

The tutorial focuses on adding an **Remote Notifications** using Firebase for both Android and iOS devices. It is essential to ensure that the platform is set to iOS before proceeding with the integration steps.

## Creating a Firebase Project

- The first step involves visiting the Firebase console at [console.firebase.google.com](https://console.firebase.google.com) to create a new Firebase project.
- Users are instructed to click on the 'Create a Firebase project' button and provide a project name, such as 'test game 2'.
- After naming the project, users should click 'Continue' and select their Google Analytics account before finalizing the project creation.

## Registering Apps in Firebase

- Once the Firebase project is created, users need to register their apps, starting with Unity for both Android and iOS.
- For iOS, users must copy the bundle ID from the App Store Connect and from Google Play for Android.
- After registering both apps, users download the required configuration files: Google services.json for Android and Google service info.plist for iOS.

## Importing Configuration Files into Unity

- Users are guided to open their Unity project and import the downloaded configuration files by dragging and dropping them into the assets root folder.
- Once the files are imported, users proceed to download the Firebase Unity SDK from Firebase and import the Firebase messaging package into Unity.

## Setting Up Firebase Messaging in Unity

- After importing the Firebase messaging package, users must resolve any dependencies through the External Dependency Manager < Android Resolver < Resolve.

- After this create an empty game object named 'Remote Notifications' and add the necessary scripts for remote notifications [as mentioned in the video above].
- Users are instructed to drag and drop the remote notification script into the project, completing the Unity setup for Firebase messaging.

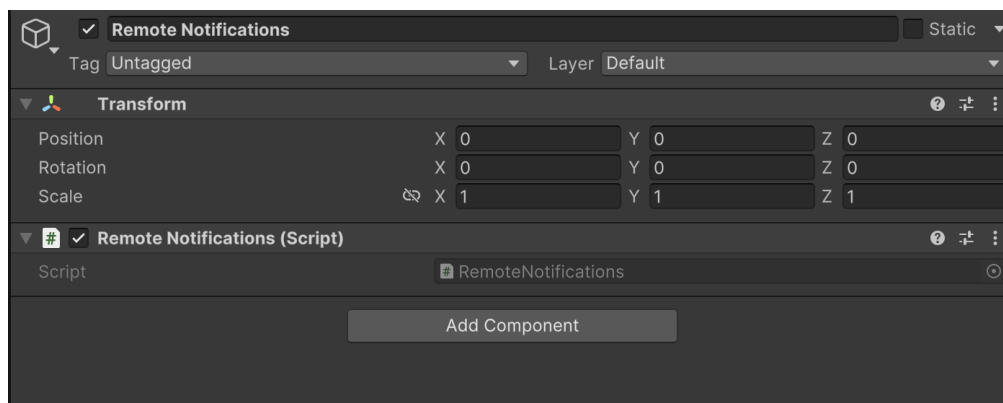
## Configuring Firebase Project Settings

- Users navigate back to the Firebase console to access the project settings for both Android and iOS apps, ensuring that the correct package name and app ID are displayed.
- For iOS, users must provide their Team ID and App Store ID, which can be retrieved from the App Store Connect.
- Once all information is filled in, users save the settings and proceed to configure cloud messaging for iOS by uploading the APN authentication key.

## Creating the APN Authentication Key

- To create the APN authentication key, users visit the Apple Developer website and generate a new key, ensuring to check the Apple Push Notification Services checkbox.
- After configuring the key settings, users download the key and upload it to Firebase, along with the Key ID, completing the setup for remote notifications.
- The setup is confirmed as complete, and users can now proceed to test the notifications.

## Remote Notifications Script



# In App Update

## Introduction

The video begins with an overview of the process of integrating **In-App updates** for Android applications.

## Downloading the Unity Package

- The first step involves navigating to the tools section within the mobile monetization pro integration tool.
- Users are instructed to select the in-app update integration option and to download the latest Android in-app update Unity package.
- After downloading, the package must be installed in the Unity project.

## Importing Required Packages

- Once the Unity package is installed, the next step is to import required scripts into the project.
- If the external dependency manager is already present, users can skip its installation and proceed to import the Google Play plugins along with the in-app update.

## Setting Up the In-App Update Manager

- The next step involves creating an empty game object, named in-app update, within the Unity hierarchy.
- Users should reset the transform of this game object and then drag and drop the in-app update manager script onto it.
- After setting up the manager script, the project is ready for testing on a real Android device.

## Configuring Project Settings

- Before testing, it is essential to configure the project settings, specifically the version and bundle version code.
- Users are advised to ensure that the package name and game name are correctly set before proceeding to the Google Play Console.

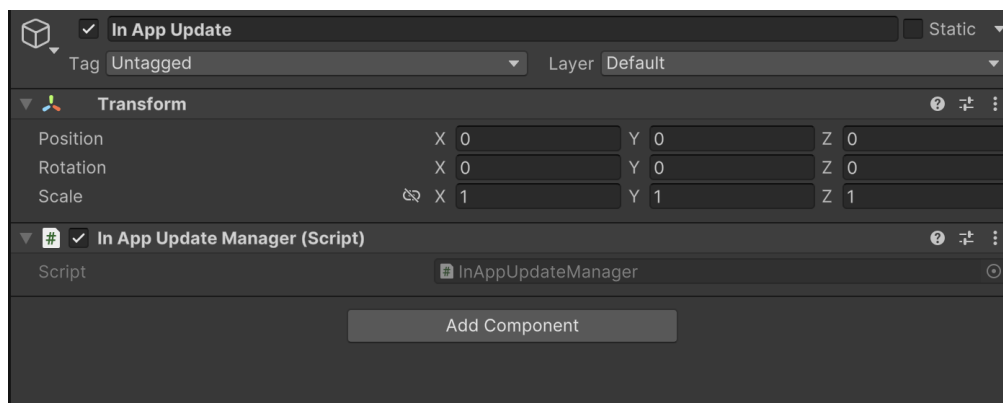
## Creating a Release in Google Play Console

- The video guides users to access the Google Play dashboard where a test environment has already been prepared.
- Users should create a new release and uploading the new bundle.

## Testing the In-App Update

- To test the in-app update functionality, a new version of the game must be uploaded to the Google Play Console.
- The version number should be updated, and release notes should be added for clarity before publishing.
- Once published, users can begin testing the update feature by opening the game and observing the update notification.

## In App Update Script



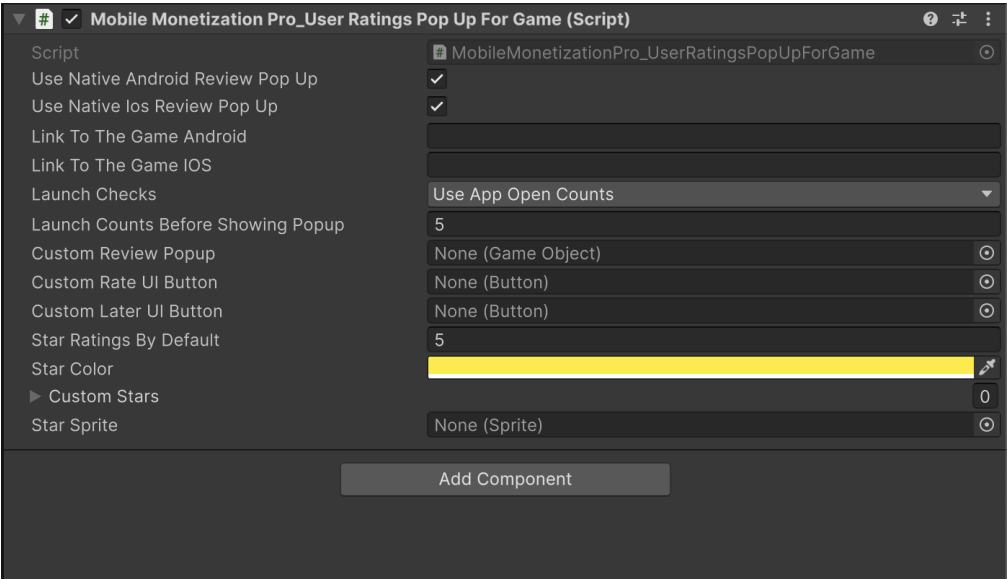
# Rate Game Popup

## Introduction

In this video, you'll learn how to integrate **Rate Game Popup** into your Unity mobile game. We will cover the following key concepts:

- **Native Android Rate Game Popup:** Learn how to create and display a native Android rate popup directly within your game.
- **iOS Native Rate Game Popup:** Follow a step-by-step guide to setting up a native iOS rate popup in your Unity project.
- **Custom Rate Game Popup Formats:** Option for customizable popup and how to implement.
- **Testing on Android and iOS:** See how to test the Rate Game Popup on real Android and iOS devices to ensure everything works perfectly.

## User Rate Game Popup



Fields	Info
UseNativeAndroidReviewPopUp	Determines whether to use the native Android review popup for rating.

UseNativeIosReviewPopUp	Determines whether to use the native iOS review popup for rating.
LinkToTheGameAndroid	Link to the game on the Android store for users to open.
LinkToTheGameIOS	Link to the game on the iOS App Store for users to open.
Options	Determines the launch condition for showing the review popup:       • UseAppOpenCounts — Use the app open counts to determine when to show the review popup.     • UseSessionCounts — Use session counts to determine when to show the review popup.
LaunchCountsBeforeShowingPopup	Number of times the game should be opened before showing the review popup.
CustomReviewPopup	Custom review popup UI GameObject.
CustomRateUIButton	Custom button to rate the game.
CustomLaterUIButton	Custom button to dismiss the review prompt.
StarRatingsByDefault	Default star rating value to be displayed.
StarColor	Color used to change the star ratings.
CustomStars	Array of custom star GameObjects to show the rating system.
StarSprite	Sprite to be activated when a star is clicked.

# Internet Connectivity Check

## Introduction

In this video, you'll learn how to integrate **Rate Game Popup** into your Unity mobile game. We will cover the following key concepts:

- Setting up a No internet connectivity checker script for a game, which alerts players when they lose internet connection.
- When the player is disconnected, a message indicating no internet connection is displayed.
- Once the player reconnects to the internet, a custom function can be triggered to reinitialize the SDK and reload ads in the game.

## Creating the No Internet Connectivity Popup

- To implement the no internet connectivity checker, an empty game object is created and named accordingly.
- The transform of this game object is reset, and a component called 'internet connectivity check' is added to perform the connectivity checks.

## Displaying the No Internet Popup

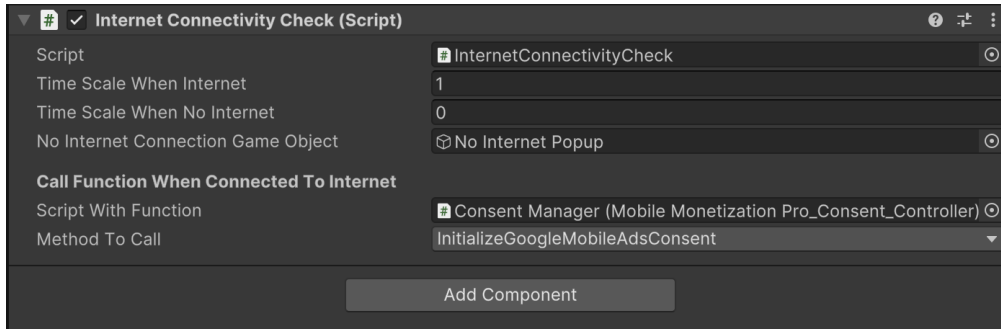
- A no internet connection popup is created within the canvas and is activated when the player loses internet connectivity.
- This popup is initially set to be deactivated and is linked to the game object to display the appropriate message when needed.
- The functionality includes defining actions that occur when the player reconnects to the internet, such as re-enabling ads in the game.
- Custom functions can be called by dragging the relevant MonoBehaviour script into the setup and selecting the desired method to invoke.

## Building and Testing the Game

- After setting up the connectivity checker script.

- Build and run on an Android device, where it is confirmed that no ads are displayed when there is no internet connection.
- Upon connecting to Wi-Fi, the game successfully displays ads, demonstrating that the reinitialization works as intended.

## Internet Connectivity Check



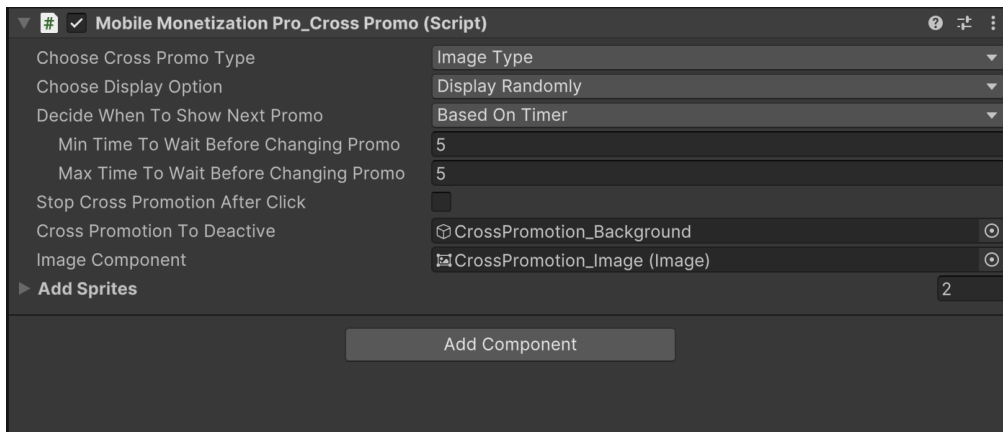
Fields	Info
TimeScaleWhenInternet	Time scale when the internet connection is active (1 = normal speed).
TimeScaleWhenNoInternet	Time scale when there is no internet connection (0 = paused).
NoInternetConnectionGameObject	The GameObject to display when there is no internet connection.
scriptWithFunction	The MonoBehaviour script that contains the method to invoke when internet is restored.
methodName	The name of the method to invoke from the scriptWithFunction when internet is restored.

# Cross-Promotion

## Introduction

The tutorial focuses on implementing **Cross-Promotion** within a game, providing step-by-step instructions on how to set it up.

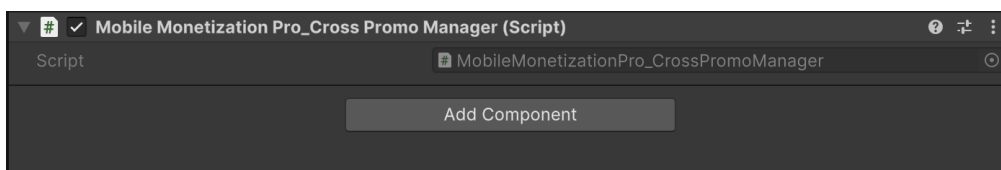
## Cross-Promo Script



Fields	Info
ChooseCrossPromoType	Select the type of cross-promotion to display.
videoPlayer	Video player component used to play video cross-promotions.
ChooseDisplayOption	Choose how to display the sprites.
RawImageComponent	RawImage component for displaying video or image.
RenderTextureComponent	Render texture component for the video player.
ImageComponent	Image component for displaying image cross-promotions.
AddVideos	List of video cross-promotions to display.

AddSprites	List of image cross-promotions to display.
DecideWhenToShowNextPromo	Determine when the next promo should be displayed.
NoOfAppOpensToCheckBeforeNewPromo	Number of app opens to check before showing a new cross-promotion.
NoOfSessionsToCheckBeforeNewPromo	Number of sessions to check before showing a new cross-promotion.
MinTimeToWaitBeforeChangingPromo	Minimum time (in seconds) to wait before changing the cross-promotion.
MaxTimeToWaitBeforeChangingPromo	Maximum time (in seconds) to wait before changing the cross-promotion.
StopCrossPromotionAfterClick	If true, stop cross-promotion after the user clicks.
CrossPromotionToDeactive	GameObject to deactivate when cross-promotion ends.

## Cross-Promo Manager Script



# Firebase Analytics

## Introduction

The tutorial focuses on integrating **Firebase Analytics** into a Unity project.

## Creating a Firebase Project

- Users need to create a new Firebase project by clicking on the "Add Project" option.
- As an example - project is named "Size Shift", and users are instructed to enable Google Analytics for the project before proceeding.
- After selecting their Google account, users complete the project creation process by clicking "Create Project" and then "Continue".

## Registering the Unity App

- Next, users must register their Unity application as an Unity Android app within the Firebase console.
- To do this, they need to enter the package name from their Unity project settings and provide an app nickname, which is example - "Size Shift".
- Once registered, users are prompted to download the Google services JSON file, which must be correctly placed in their Unity project directory.

## Downloading and Installing Firebase SDK

- Users are instructed to download the Firebase Unity SDK and extract the contents of the zip file.
- After extraction, the Firebase Analytics Unity package should be imported into the Unity project by dragging and dropping it into the project window.

## Configuring Firebase in Unity

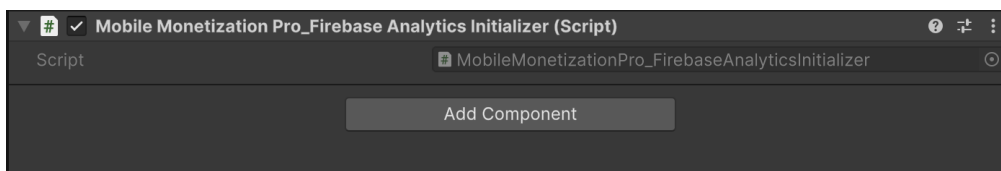
- Within Unity, users must navigate to the Tools menu and select the Mobile Monetization Pro integration tool to choose Firebase Analytics.
- After confirming that the Firebase SDK is installed, users need to click on "Import Required Scripts" and close the integration window.

- They need to create an empty game object named "Firebase Analytics" and drag the Firebase initializer script into it.

## Resolving Errors During Integration

- Upon entering play mode, users may encounter an error indicating that the Firebase CPP bundle cannot be opened due to verification issues.
- To resolve this, users need to adjust their system settings by navigating to Security and allowing the Firebase CPP app [As shown in the video above].
- After allowing the app, users can exit play mode, clear the console, and re-enter play mode to verify successful integration.

## Firebase Analytics Initializer Script



# Contact

## [Discord Support](#)

For faster assistance, join our **Discord channel**.

## [Email Support](#)

For any questions about our assets or inquiries before purchase, contact us by mail.